SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANSWERS

Course: Artificial Intelligence | Course Code: CSA17 | CO2

Name: YUVANSHANKAR M **Reg. No.:** 1912260450

Question 1: AI-Powered Emergency Medicine Delivery Drones

i. Greedy Best-First Search

Greedy Best-First Search selects the next location with the smallest heuristic value, h(n), which in this case is the straight-line distance to the medicine delivery point. It may quickly choose a route that appears closest to the destination. This is useful when time is critical and a quick decision is needed. However, it ignores the cost already spent, may select a route with severe weather or difficult terrain, and does not guarantee the shortest or safest route. In an uncertain environment, partial updates can make its chosen path outdated.

ii. A* Search

A* evaluates each candidate using f(n) = g(n) + h(n). Here, g(n) is the actual travel cost accumulated so far, including time, terrain and weather penalties, while h(n) estimates the remaining distance or cost to the destination. By combining past cost and estimated remaining cost, A* avoids choosing a route only because it looks close. If the heuristic is admissible and costs are represented accurately in a stable graph, A* can find an optimal route. In a changing environment, it must update costs and re-plan when new information arrives.

iii. Impact of heuristic accuracy

A good heuristic guides the search toward the goal and reduces the number of explored locations. If straight-line distance is a poor estimate because of rivers, blocked roads, storms or restricted flight zones, Greedy Best-First Search may be misled strongly because it relies mainly on h(n). A* is also affected, but g(n) helps account for known costs. For A* optimality, the heuristic should not overestimate the true remaining cost; a consistent heuristic is also helpful for efficient graph search.

iv. Dynamic blocked paths

When a path becomes blocked, the drone's map and edge costs must be updated. A route that was previously best may no longer be usable, so the algorithm needs to re-plan from the current position. Repeatedly restarting A* can consume time; incremental methods such as Lifelong Planning A* or D* Lite can reuse previous search information. The drone should also use safety rules and verified sensor updates before entering uncertain areas.

v. Recommendation

A* with real-time cost updates is the best general choice when both delay and risk matter, because it considers the route cost already incurred as well as the estimated remaining cost. For fast operation, use an admissible distance-based heuristic adjusted for terrain and weather, and re-plan whenever reliable updates report a blocked path. If the environment changes very frequently, an incremental replanning algorithm such as D* Lite may be more practical. Safety constraints should be treated as hard restrictions, not merely small cost penalties.

Question 2: Smart-City Traffic Signal Optimization

Formulation as an optimization problem

Represent the traffic network as intersections and roads. The decision variables are signal phases, green-light durations and their schedules. The objective is to minimize a weighted total cost such as $J = \alpha W + \beta F + \gamma C$, where W is total vehicle waiting time, F is fuel consumption, C is congestion, and α, β and γ express the city's priorities. Constraints include safe phase ordering, minimum and maximum green times, pedestrian crossing times, emergency-vehicle priority and coordination between nearby intersections. Traffic measurements update the problem continuously.

i. Hill Climbing

Hill Climbing begins with a signal plan and repeatedly changes one or more timings. It evaluates each neighbouring plan and keeps a change if it improves the objective. It is simple and can make quick local adjustments. However, it may stop at a locally good plan even when a much better plan exists elsewhere. It also depends on the starting solution and may respond poorly when traffic changes suddenly.

ii. Local maxima, plateaus and ridges

A local optimum occurs when nearby timing changes do not improve the current plan, although a better overall plan exists. A plateau occurs when many neighbouring plans have almost the same score, so the algorithm has little direction. A ridge occurs when improvement requires a coordinated change across several intersections, while changing one signal at a time appears unhelpful. In practice, changing only one intersection may move a queue to the next junction rather than reduce network-wide congestion.

iii. Simulated Annealing or Genetic Algorithms

Simulated Annealing sometimes accepts a worse solution early in the search, with a probability that decreases as the temperature parameter falls. This helps it escape local optima and gradually focus on better plans. A Genetic Algorithm maintains a population of candidate signal plans and uses selection, crossover and mutation to create new plans. It explores several alternatives in parallel but may require more computation and careful parameter tuning. Both approaches can be combined with real-time local adjustments.

iv. Recommendation

A hybrid approach is suitable: use a Genetic Algorithm or Simulated Annealing offline or periodically to find strong network-wide timing plans, then use fast local optimization and traffic feedback for real-time adaptation. A Genetic Algorithm is useful when the search space is very large and many intersections must be coordinated; Simulated Annealing may be simpler when a single candidate can be evaluated efficiently. In either case, safety constraints must be enforced and performance should be tested in simulation before deployment.

Question 3: Autonomous Mars Rover

i. Why online search is required

Offline search assumes the environment and map are known before planning. A Mars rover has only partial information and may discover rocks, craters, loose soil or unexpected slopes as it moves. Online search allows the rover to sense its surroundings, choose an action, observe the result and update its plan. This sense–act–learn cycle is essential when the complete map is unavailable.

ii. Partial observability and dynamic conditions

Sensors have limited range and may contain noise; terrain behind obstacles may be unknown. A rover can also face changing illumination, communication delays, wheel slip and uncertain soil strength. Therefore, it cannot rely on a perfectly accurate map. It must represent uncertainty, maintain a margin from hazards and avoid actions whose risk cannot be assessed adequately.

iii. Building and updating the internal model

The rover combines camera images, depth or range measurements, wheel odometry and other available sensor data to build a local map. It labels cells or regions as traversable, blocked or uncertain and estimates the cost of movement based on slope, roughness and energy use. After each movement, it compares predicted and observed results, updates its position estimate and map, and replans if new obstacles are detected. Uncertain regions should be explored cautiously.

iv. Comparison of search strategies

Uninformed methods such as breadth-first search do not use goal-distance information and can explore many irrelevant states. Depth-first search may use less memory in some cases but can follow an unsafe or unproductive route. Informed methods such as A* use a heuristic to guide planning and can find efficient routes when a useful map is available. Online search differs because it repeatedly acts and learns while exploring, rather than completing all planning before movement.

v. Recommended approach

Use an online, receding-horizon planner: plan over the currently mapped area, move a short safe distance, collect new sensor data and replan. A* can be used on the known local map with a heuristic that estimates distance to the sample site, while risk and energy costs are included in the path cost. A safety layer should reject paths too close to hazards or through uncertain terrain. This balances adaptability, efficiency and rover protection.

Question 4: University Exam Timetabling as a CSP

Formulation: variables, domains and constraints

A Constraint Satisfaction Problem (CSP) consists of variables, domains and constraints. Variables: one variable for each exam, representing its assigned time slot and hall. Domains: the available time slots and halls that can accommodate each exam, after considering duration, capacity and faculty availability. Constraints: (1) exams attended by the same student must not overlap; (2) a hall cannot host two exams at the same time; (3) hall capacity and accessibility must meet student needs; (4) prerequisite ordering must be respected; (5) faculty must be available; and (6) special accommodations must be provided. The goal is to find an assignment satisfying all hard constraints, while minimizing preferences or inconvenient gaps if possible.

i. Backtracking search

Backtracking assigns a value to one exam variable at a time. After each assignment, it checks whether the current partial timetable violates a constraint. If it does, the algorithm undoes the latest assignment and tries another slot or hall. Search can be improved by choosing the most constrained exam first (Minimum Remaining Values), selecting the exam that constrains others most when there is a tie (degree heuristic), and trying the least-constraining value first.

ii. Constraint propagation and forward checking

After assigning an exam, forward checking removes incompatible slots or halls from the domains of neighbouring exams. If any unassigned exam has an empty domain, the current choice cannot lead to a solution and the algorithm backtracks immediately. This avoids exploring many combinations that would later fail. Stronger propagation methods can repeatedly apply constraints to reduce domains before making the next assignment.

iii. Real-world challenges and scalability

Large universities may have thousands of students, many courses, limited halls, shared faculty and special requirements. Preferences may conflict, and enrolment data can change. A practical strategy is to combine CSP backtracking with MRV, degree heuristic, least-constraining values and forward checking. For large cases, decompose the timetable by campus, programme or time period, then reconcile shared constraints. Local search can improve the quality of an already feasible timetable, but hard constraints must remain satisfied.

Question 5: Minimax and Alpha-Beta Pruning in Game AI

i. Minimax model

Minimax represents the game as a tree of states and legal moves. The AI is the maximizing player and attempts to choose a move that gives the highest score, assuming the human opponent is the minimizing player and chooses moves that reduce that score. Terminal states receive utility values such as win, draw or loss. When the full tree is too large, a depth-limited search evaluates the leaf states with a heuristic evaluation function.

ii. Computational challenges

If each state has b legal moves and the search depth is d, the number of nodes can grow approximately as O(b^d). Even a moderate branching factor makes deep search expensive. Real-time games also have strict response deadlines, large state descriptions and actions whose consequences may extend beyond the search horizon. A fixed time limit therefore requires selective or depth-limited search.

iii. Alpha-Beta pruning

Alpha-Beta pruning produces the same minimax decision as ordinary minimax when it searches the same tree and evaluation values, but it avoids exploring branches that cannot affect the final choice. Alpha (α) is the best value found so far for the maximizing player; beta (β) is the best value found so far for the minimizing player. At a maximizing node, α is updated when a higher value is found; at a minimizing node, β is updated when a lower value is found. When $\alpha \geq \beta$, the remaining children of that node can be pruned because the opponent already has a choice that makes the branch irrelevant. Good move ordering increases pruning. In the best case, the effective complexity can approach O(b^(d/2)), allowing deeper search in the same time.

iv. Evaluation function

A practical heuristic evaluation can be written as E(s) = w1M + w2R + w3O + w4T – w5D, where M is the difference in military or unit strength, R is resource advantage, O is objective or territory control, T is technology or strategic progress, and D is damage or vulnerability to the AI's side. The weights w1–w5 reflect game rules and should be tuned through testing. Terminal win/loss values should outweigh ordinary heuristic scores. The function should be fast to compute and should avoid double-counting correlated factors.

v. Recommended real-time strategy

Use iterative-deepening minimax with Alpha-Beta pruning, good move ordering, a transposition table and a strict time budget. The system first finds a legal move at shallow depth, then searches deeper while time remains, retaining the best completed result. A carefully designed evaluation function estimates positions at the depth limit. This provides strong decisions without requiring a complete search of the game tree. Safety checks and a legal fallback move ensure that the AI can still act when the time limit is reached.